

Take-Home Exercise

The Coordinator

Backend / Distributed Systems

Time expectation: We expect this to take 3–5 hours. Please don't spend more than a weekend on it. If you find yourself going deep on infrastructure or polish, stop and write a note instead. We care far more about your reasoning than about completeness.

What to send back: A working service in any language, plus a short design note (markdown is fine, roughly 1–2 pages). The design note matters as much as the code.

How we read it: We are not counting features. We are reading for judgment: did you understand the problem you were actually given, and can you defend the calls you made?

The situation

You're joining a team that operates a fleet of worker instances. Each worker processes jobs from a shared queue. Some jobs touch a shared external resource, such as a billing ledger, a document store, or an inventory record, where it is unacceptable for two workers to act on the same entity at the same time. Concurrent action causes real, visible corruption: double charges, lost writes, duplicated shipments.

The team already has a coordination service available (you can assume something Redis-like, or etcd/ZooKeeper-like; your choice, state your assumption). The previous owner left behind a one-paragraph design doc that says, essentially:

Inherited design note (verbatim)

"Before a worker touches the resource, it acquires a lock on the entity key with a TTL. If it gets the lock, it does the work, then releases. The TTL means we don't deadlock if a worker dies. We've been running this for months and it's basically fine."

Your job: implement this coordination correctly, and write the design note the previous owner didn't.

What we want you to build

1. **A lock/coordination component** that a worker calls before and after touching the protected resource. Mutual exclusion on a given entity key is the core requirement.
2. **A small simulation or test harness** that exercises your component with multiple concurrent workers contending for the same entity, and demonstrates that the protected resource stays consistent.
3. **A design note** (see the required sections below).

You may use libraries for the coordination primitive itself (a Redis client, an etcd client, an in-memory stand-in). We're not asking you to reimplement Raft. We **are** asking you to reason about what the primitive does and does not give you.



Operating conditions

These are the real conditions the service runs under. Your test harness should be able to produce them; your design note should account for them.

- **Workers are not reliable narrators of their own liveness.** A worker can stall (a long GC pause, a blocked syscall, a noisy-neighbor CPU steal) for longer than you'd expect. From the outside it looks dead; then it wakes up and keeps going exactly where it left off, with no awareness that time passed.
- **The network between a worker and the resource is not instantaneous or ordered.** A request a worker sends can sit in flight and be delivered late, after the worker that sent it believed it had moved on.
- **The coordination service measures time, and time is imperfect.** TTLs expire according to a clock. Clocks drift; occasionally they're corrected and step. The expiry you observe and the expiry the worker assumes may not be the same instant.
- **Jobs are not all the same length.** Most complete in well under a second. A minority, the ones that matter, run long enough to brush up against any TTL you'd consider reasonable. Lowering the TTL to be safe makes those long jobs lose their lock mid-flight. Raising it makes a dead worker hold the resource hostage. There is no value that escapes both.

Required sections in your design note

We will read these closely. Brevity is fine; evasion is not.

- **The guarantee.** State precisely what your design guarantees about the protected resource, and under exactly which assumptions. If your guarantee depends on something being true (clock bounds, max pause duration, network behaviour), name it.
- **The failure you can't prevent at the lock.** Describe the worst thing that can still happen, and where in the system it gets caught, if it gets caught.
- **The TTL decision.** Tell us how you'd choose the timeout, what breaks at the extremes, and what you actually do about the long-job-versus-dead-worker tension above. "It depends" is acceptable only if you say what it depends on.
- **What you'd do with more time / in production.** Where does your submission cut corners on purpose?

Ground rules

- **Use whatever tools you normally use,** including AI assistants. We assume you will. The exercise is built so that the tools help you write the code but don't make the judgment calls for you. That part is the interview.
- **If a requirement seems under-specified, that may be deliberate.** Make a reasonable assumption, write it down, and move on. We're more interested in which ambiguities you noticed than in you guessing what we 'wanted.'
- **A smaller submission with a sharp design note beats a large one that hand-waves.** If you run out of time, leave the code where it is and explain in the note what you'd do next and why.

Questions about scope or setup are welcome; email us. Questions of the form "which approach do you want?" we'll likely turn back to you, because the answer is part of what we're evaluating.